AI-ASSISTED CODING

ASSIGNMENT 14.3

NAME:SIDDARTHA

HALL:2303A52062

Task 1: AI-Assisted Personal Portfolio Website

Scenario

A student wants to showcase academic projects, technical skills, and contact information through a personal portfolio website. AI tools are used to speed up development while maintaining quality and customization.

Tasks

1. Use GitHub Copilot to generate a semantic HTML structure with sections:

o Home

o About

o Projects

o Contact

2. Ask Copilot to suggest responsive CSS using Flexbox or Grid.

3. Customize AI-generated CSS to add:

o Hover effects on project cards

o Smooth scrolling navigation

Expected Outcome

• Clean and well-structured portfolio website

• Responsive layout for desktop and mobile screens

• Interactive project cards

PROMPT

Create a semantic personal portfolio website using HTML with sections Home, About, Projects, and Contact. Add responsive CSS using Flexbox/Grid, smooth scrolling navigation, and hover effects on project cards.

CODE

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Task 1 - Personal Portfolio</title>
  <style>
    :root {
      --bg: #f4f7fb;
      --surface: #ffffff;
      --text: #1e2a38;
      --muted: #5b6b7d;
      --primary: #0f766e;
      --primary-soft: #d5f5f2;
      --border: #dbe5ef;
      --shadow: 0 10px 24px rgba(17, 34, 68, 0.08);
    }

    * {
      box-sizing: border-box;
    }

    html {
      scroll-behavior: smooth;
    }

    body {
      margin: 0;
```

```
  font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
  background: linear-gradient(180deg, #eef3f9 0%, #f8fbff 100%);
  color: var(--text);
}

header {
  position: sticky;
  top: 0;
  z-index: 10;
  background: rgba(255, 255, 255, 0.95);
  border-bottom: 1px solid var(--border);
  backdrop-filter: blur(6px);
}

nav {
  max-width: 1100px;
  margin: 0 auto;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 14px 18px;
}

.brand {
  font-weight: 700;
  color: var(--primary);
  letter-spacing: 0.3px;
}
```

```
.nav-links {
  display: flex;
  gap: 16px;
}

.nav-links a {
  text-decoration: none;
  color: var(--text);
  font-weight: 600;
}

.nav-links a:hover {
  color: var(--primary);
}

main {
  max-width: 1100px;
  margin: 0 auto;
  padding: 22px 18px 50px;
  display: grid;
  gap: 18px;
}

section {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 14px;
```

```
  padding: 22px;
  box-shadow: var(--shadow);
}

#home h1 {
  margin: 0;
  font-size: 2rem;
}

#home p {
  color: var(--muted);
}

.project-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 14px;
}

.project-card {
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 16px;
  background: #fff;
  transition: transform 0.25s ease, box-shadow 0.25s ease, border-color 0.25s ease;
}

.project-card:hover {
```

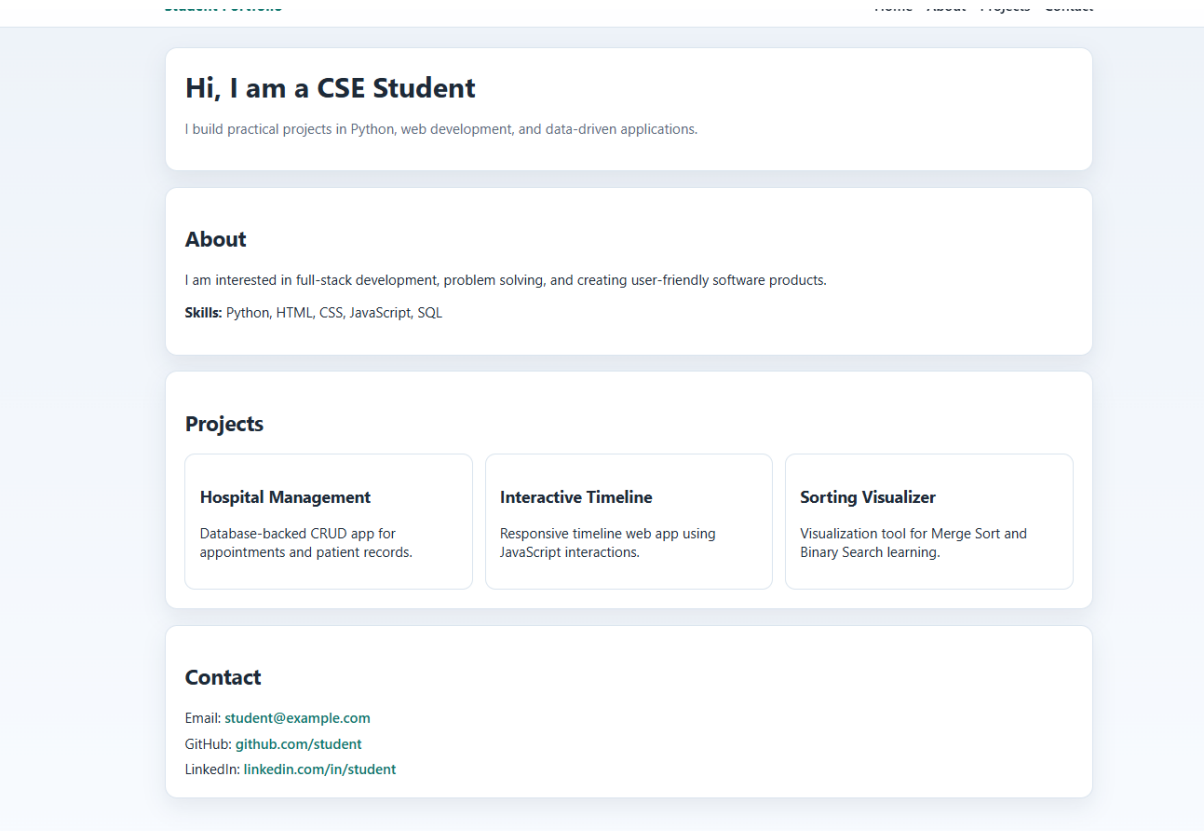
```
  transform: translateY(-6px);
  box-shadow: 0 12px 24px rgba(15, 118, 110, 0.2);
  border-color: var(--primary);
}

.contact-list {
  margin: 0;
  padding: 0;
  list-style: none;
  display: grid;
  gap: 8px;
}

.contact-list a {
  color: var(--primary);
  text-decoration: none;
  font-weight: 600;
}

@media (max-width: 700px) {
  nav {
    flex-direction: column;
    align-items: flex-start;
    gap: 10px;
  }

  .nav-links {
    flex-wrap: wrap;
```

```
    }

    #home h1 {
      font-size: 1.6rem;
    }
  }
 </style>
</head>
<body>
 <!-- Simple Prompt: Generate a semantic personal portfolio website with Home, About, Projects, Contact sections; responsive layout; smooth scrolling; and project card hover effects. -->

 <header>
  <nav>
    <div class="brand">Student Portfolio</div>
    <div class="nav-links">
      <a href="#home">Home</a>
      <a href="#about">About</a>
      <a href="#projects">Projects</a>
      <a href="#contact">Contact</a>
    </div>
  </nav>
 </header>

 <main>
  <section id="home">
    <h1>Hi, I am a CSE Student</h1>
```

```
    <p>I build practical projects in Python, web development, and data-driven
applications.</p>
  </section>

  <section id="about">
    <h2>About</h2>
    <p>I am interested in full-stack development, problem solving, and creating user-
friendly software products.</p>
    <p><strong>Skills:</strong> Python, HTML, CSS, JavaScript, SQL</p>
  </section>

  <section id="projects">
    <h2>Projects</h2>
    <div class="project-grid">
      <article class="project-card">
        <h3>Hospital Management</h3>
        <p>Database-backed CRUD app for appointments and patient records.</p>
      </article>
      <article class="project-card">
        <h3>Interactive Timeline</h3>
        <p>Responsive timeline web app using JavaScript interactions.</p>
      </article>
      <article class="project-card">
        <h3>Sorting Visualizer</h3>
        <p>Visualization tool for Merge Sort and Binary Search learning.</p>
      </article>
    </div>
  </section>
```

```
  <section id="contact">
    <h2>Contact</h2>
    <ul class="contact-list">
      <li>Email: <a href="mailto:student@example.com">student@example.com</a></li>
      <li>GitHub: <a href="https://github.com/" target="_blank"
rel="noreferrer">github.com/student</a></li>
      <li>LinkedIn: <a href="https://linkedin.com/" target="_blank"
rel="noreferrer">linkedin.com/in/student</a></li>
    </ul>
  </section>
 </main>
</body>
</html>
```

OUTPUT



Task 2: AI-Generated Restaurant Landing Page

Scenario

A local restaurant requires a simple yet attractive landing page to display its offerings. The developer uses AI assistance to rapidly generate UI components.

Tasks

1. Use Copilot to create a navigation bar with links:

o Home

o Menu

o Gallery

o Contact

2. Generate a menu section styled with CSS cards.

3. Implement a JavaScript-based image slider for the gallery using Copilot suggestions.

4. Customize animations or transitions for better user experience.

Expected Outcome

- **Fully functional restaurant landing page**
- **Responsive navigation bar**
- **Working image slider with smooth transitions**

PROMPT

Build a responsive restaurant landing page with a navbar (Home, Menu, Gallery, Contact), a menu section using CSS cards, and a JavaScript image slider with smooth transitions and basic animations.

CODE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Task 2 - Restaurant Landing Page</title>
```

```
<style>
  :root {
    --bg: #fff8ef;
    --text: #2a2018;
    --primary: #c2410c;
    --card: #ffffff;
    --border: #f1d5bd;
    --shadow: 0 8px 20px rgba(116, 66, 35, 0.15);
  }

  * {
    box-sizing: border-box;
  }

  body {
    margin: 0;
    font-family: "Trebuchet MS", "Segoe UI", sans-serif;
    color: var(--text);
    background: radial-gradient(circle at top right, #ffe7cf, #fff8ef 50%);
  }

  .navbar {
    position: sticky;
    top: 0;
    z-index: 5;
    background: rgba(255, 248, 239, 0.95);
    border-bottom: 1px solid var(--border);
    display: flex;
```

```
  justify-content: space-between;
  align-items: center;
  padding: 14px 20px;
}

.logo {
  font-weight: 800;
  color: var(--primary);
}

.links {
  display: flex;
  gap: 16px;
}

.links a {
  text-decoration: none;
  color: var(--text);
  font-weight: 600;
  transition: color 0.2s ease;
}

.links a:hover {
  color: var(--primary);
}

section {
  max-width: 1100px;
```

```
  margin: 0 auto;
  padding: 26px 18px;
}

.hero {
  text-align: center;
  padding-top: 40px;
}

.menu-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
  gap: 14px;
}

.menu-card {
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 14px;
  box-shadow: var(--shadow);
  transition: transform 0.25s ease;
}

.menu-card:hover {
  transform: translateY(-4px);
}
```

```
.slider {
  position: relative;
  max-width: 700px;
  margin: 8px auto 0;
  border-radius: 12px;
  overflow: hidden;
  border: 1px solid var(--border);
  box-shadow: var(--shadow);
  background: #fff;
}

.slide {
  display: none;
  width: 100%;
  height: 300px;
  object-fit: cover;
  transition: opacity 0.5s ease;
}

.slide.active {
  display: block;
  animation: fadeIn 0.5s ease;
}

.slider-controls {
  display: flex;
  justify-content: center;
  gap: 8px;
```

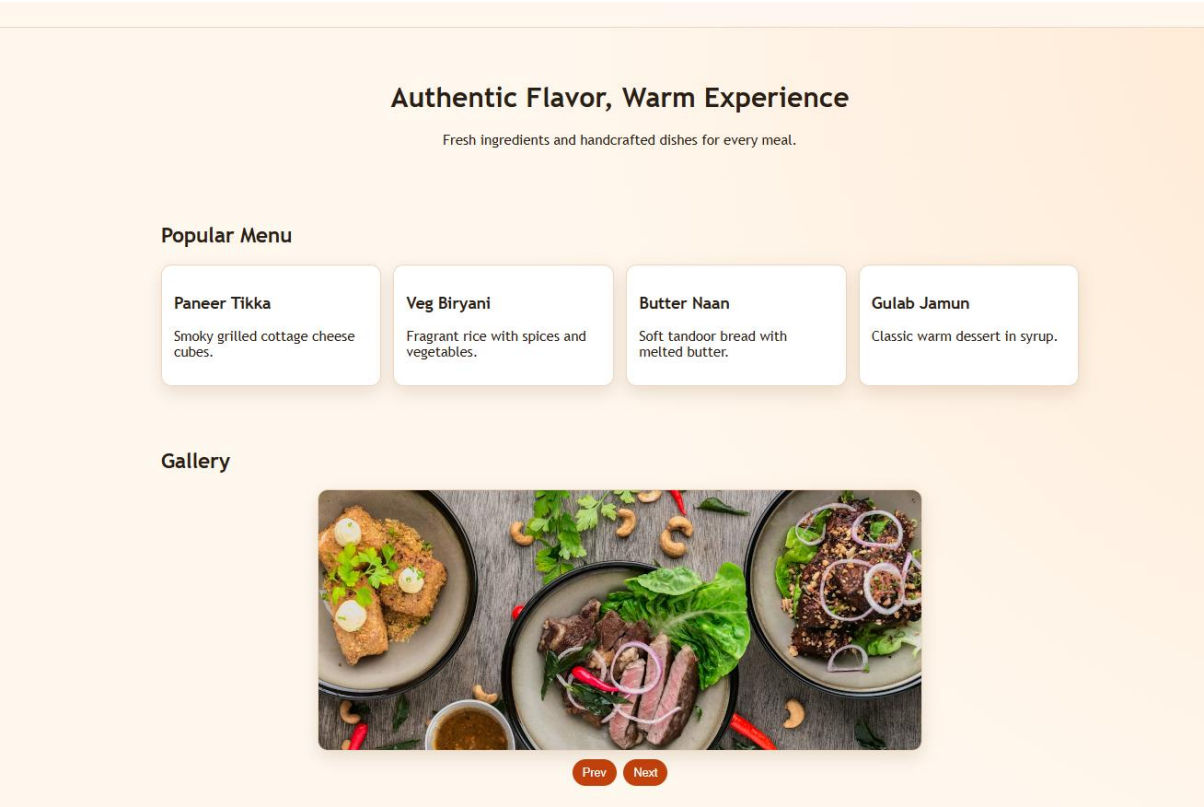
```
  margin-top: 10px;
}

.slider-controls button {
  border: 0;
  border-radius: 999px;
  padding: 8px 12px;
  background: var(--primary);
  color: #fff;
  cursor: pointer;
  transition: transform 0.2s ease;
}

.slider-controls button:hover {
  transform: scale(1.05);
}

@keyframes fadeIn {
  from { opacity: 0.3; }
  to { opacity: 1; }
}

@media (max-width: 720px) {
  .navbar {
    flex-direction: column;
    align-items: flex-start;
    gap: 8px;
  }
```

```
    .links {
      flex-wrap: wrap;
    }

    .slide {
      height: 220px;
    }
  }
 </style>
</head>
<body>
 <!-- Simple Prompt: Build a responsive restaurant landing page with Home/Menu/Gallery/Contact nav links, CSS menu cards, and a JavaScript image slider with smooth transitions. -->

 <nav class="navbar">
  <div class="logo">Spice Garden</div>
  <div class="links">
    <a href="#home">Home</a>
    <a href="#menu">Menu</a>
    <a href="#gallery">Gallery</a>
    <a href="#contact">Contact</a>
  </div>
 </nav>

 <section id="home" class="hero">
  <h1>Authentic Flavor, Warm Experience</h1>
  <p>Fresh ingredients and handcrafted dishes for every meal.</p>
```

```
  </section>


  <section id="menu">
    <h2>Popular Menu</h2>
    <div class="menu-grid">
      <article class="menu-card"><h3>Paneer Tikka</h3><p>Smoky grilled cottage cheese cubes.</p></article>
      <article class="menu-card"><h3>Veg Biryani</h3><p>Fragrant rice with spices and vegetables.</p></article>
      <article class="menu-card"><h3>Butter Naan</h3><p>Soft tandoor bread with melted butter.</p></article>
      <article class="menu-card"><h3>Gulab Jamun</h3><p>Classic warm dessert in syrup.</p></article>
    </div>
  </section>


  <section id="gallery">
    <h2>Gallery</h2>
    <div class="slider">
      <img class="slide active" src="https://images.unsplash.com/photo-1504674900247-0877df9cc836?auto=format&fit=crop&w=1200&q=80" alt="Restaurant dish 1" />
      <img class="slide" src="https://images.unsplash.com/photo-1517248135467-4c7edcad34c4?auto=format&fit=crop&w=1200&q=80" alt="Restaurant dish 2" />
      <img class="slide" src="https://images.unsplash.com/photo-1540189549336-e6e99c3679fe?auto=format&fit=crop&w=1200&q=80" alt="Restaurant dish 3" />
    </div>
    <div class="slider-controls">
      <button id="prevBtn" type="button">Prev</button>
      <button id="nextBtn" type="button">Next</button>
    </div>
```

```
  </section>

  <section id="contact">
    <h2>Contact</h2>
    <p>Call: +91 98765 43210 | Email: hello@spicegarden.com</p>
  </section>

  <script>
    const slides = document.querySelectorAll('.slide');
    let index = 0;

    function showSlide(newIndex) {
      slides[index].classList.remove('active');
      index = (newIndex + slides.length) % slides.length;
      slides[index].classList.add('active');
    }

    document.getElementById('nextBtn').addEventListener('click', () => showSlide(index +
1));
    document.getElementById('prevBtn').addEventListener('click', () => showSlide(index -
1));

    setInterval(() => showSlide(index + 1), 3500);
  </script>
</body>
</html>
```

OUTPUT



Task 3: AI-Powered Event Registration Form

Scenario

SR University is hosting a technical fest and requires an online registration form with real-time validation to ensure accurate user input.

Tasks

1. Ask Copilot to generate an HTML form with the following fields:

o Name

o Email

o Phone Number

o Department

o Event Selection

2. Use AI assistance to style the form using CSS for readability and accessibility.

3. Implement JavaScript validation using Copilot:

o Email format validation

o Phone number length validation

o Required field checks

Expected Outcome

• User-friendly and visually appealing form

• Real-time validation messages

• Prevention of invalid submissions

PROMPT

Generate an event registration form with fields Name, Email, Phone Number, Department, and Event Selection. Style it for readability/accessibility and add real-time JavaScript validation for required fields, email format, and 10-digit phone number.

CODE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Task 3 - Event Registration Form</title>
  <style>
    :root {
      --bg: #eef6ff;
      --card: #ffffff;
      --text: #1e293b;
      --accent: #0ea5e9;
      --error: #dc2626;
      --ok: #059669;
      --border: #d9e8f7;
    }
```

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Segoe UI", Tahoma, sans-serif;
  background: linear-gradient(160deg, #e0f2fe, #f8fbff);
  color: var(--text);
  min-height: 100vh;
  display: grid;
  place-items: center;
  padding: 20px;
}

.form-card {
  width: min(720px, 100%);
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 22px;
  box-shadow: 0 12px 28px rgba(15, 23, 42, 0.1);
}

h1 {
  margin-top: 0;
  margin-bottom: 6px;
}
```

```
.sub {
  margin-top: 0;
  color: #475569;
}

.grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
}

.field {
  display: grid;
  gap: 6px;
}

.field.full {
  grid-column: 1 / -1;
}

label {
  font-weight: 600;
  font-size: 0.95rem;
}

input,
select {
```

```
  width: 100%;
  border: 1px solid #c9d9ea;
  border-radius: 10px;
  padding: 10px 12px;
  font: inherit;
}

input:focus,
select:focus {
  outline: none;
  border-color: var(--accent);
  box-shadow: 0 0 0 3px rgba(14, 165, 233, 0.15);
}

.msg {
  min-height: 18px;
  font-size: 0.84rem;
}

.msg.error {
  color: var(--error);
}

.msg.ok {
  color: var(--ok);
}

button {
```

```
    margin-top: 10px;
    border: 0;
    border-radius: 10px;
    background: var(--accent);
    color: #fff;
    font-weight: 700;
    padding: 11px 14px;
    cursor: pointer;
  }

  button:hover {
    filter: brightness(0.95);
  }

  @media (max-width: 680px) {
    .grid {
      grid-template-columns: 1fr;
    }
  }
 </style>
</head>
<body>
 <!-- Simple Prompt: Create an accessible event registration form with Name, Email, Phone, Department, Event fields and real-time JavaScript validation for required fields, email format, and 10-digit phone number. -->

 <form class="form-card" id="regForm" novalidate>
  <h1>SR University Tech Fest</h1>
  <p class="sub">Event Registration Form</p>
```

```
  <div class="grid">
    <div class="field">
      <label for="name">Name</label>
      <input id="name" name="name" type="text" placeholder="Enter full name" required
/>
      <div class="msg" id="nameMsg"></div>
    </div>

    <div class="field">
      <label for="email">Email</label>
      <input id="email" name="email" type="email" placeholder="name@example.com"
required />
      <div class="msg" id="emailMsg"></div>
    </div>

    <div class="field">
      <label for="phone">Phone Number</label>
      <input id="phone" name="phone" type="tel" maxlength="10" placeholder="10-digit
number" required />
      <div class="msg" id="phoneMsg"></div>
    </div>

    <div class="field">
      <label for="department">Department</label>
      <input id="department" name="department" type="text" placeholder="CSE / ECE /
MECH" required />
      <div class="msg" id="departmentMsg"></div>
    </div>
```

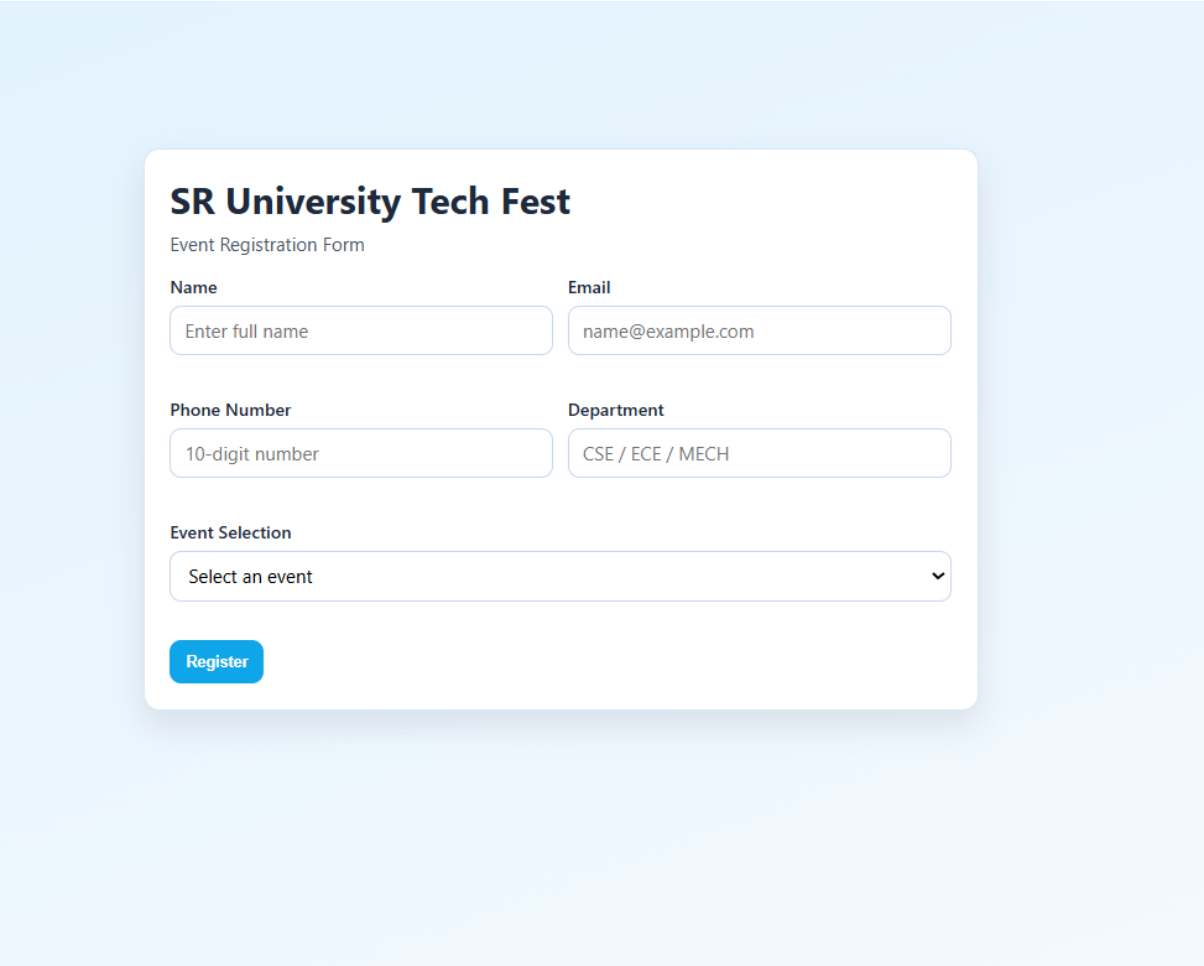
```
    <div class="field full">
      <label for="event">Event Selection</label>
      <select id="event" name="event" required>
        <option value="">Select an event</option>
        <option value="hackathon">Hackathon</option>
        <option value="paper">Paper Presentation</option>
        <option value="robotics">Robotics Challenge</option>
        <option value="quiz">Tech Quiz</option>
      </select>
      <div class="msg" id="eventMsg"></div>
    </div>
  </div>

  <button type="submit">Register</button>
</form>

<script>
  const form = document.getElementById('regForm');
  const nameInput = document.getElementById('name');
  const emailInput = document.getElementById('email');
  const phoneInput = document.getElementById('phone');
  const departmentInput = document.getElementById('department');
  const eventInput = document.getElementById('event');

  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

  function setMessage(id, text, isError) {
```

```
  const box = document.getElementById(id);
  box.textContent = text;
  box.className = isError ? 'msg error' : 'msg ok';
}

function validateRequired(input, msgId, fieldName) {
  if (!input.value.trim()) {
    setMessage(msgId, fieldName + ' is required.', true);
    return false;
  }
  setMessage(msgId, fieldName + ' looks good.', false);
  return true;
}

function validateEmail() {
  if (!validateRequired(emailInput, 'emailMsg', 'Email')) return false;
  if (!emailRegex.test(emailInput.value.trim())) {
    setMessage('emailMsg', 'Enter a valid email format.', true);
    return false;
  }
  setMessage('emailMsg', 'Valid email format.', false);
  return true;
}

function validatePhone() {
  if (!validateRequired(phoneInput, 'phoneMsg', 'Phone number')) return false;
  const value = phoneInput.value.trim();
  if (!/^\d{10}$/.test(value)) {
```

```
      setMessage('phoneMsg', 'Phone number must be exactly 10 digits.', true);
      return false;
    }
    setMessage('phoneMsg', 'Phone number is valid.', false);
    return true;
  }

  nameInput.addEventListener('input', () => validateRequired(nameInput, 'nameMsg',
'Name'));
  departmentInput.addEventListener('input', () => validateRequired(departmentInput,
'departmentMsg', 'Department'));
  eventInput.addEventListener('change', () => validateRequired(eventInput, 'eventMsg',
'Event'));
  emailInput.addEventListener('input', validateEmail);
  phoneInput.addEventListener('input', validatePhone);

  form.addEventListener('submit', (e) => {
    const okName = validateRequired(nameInput, 'nameMsg', 'Name');
    const okDept = validateRequired(departmentInput, 'departmentMsg', 'Department');
    const okEvent = validateRequired(eventInput, 'eventMsg', 'Event');
    const okEmail = validateEmail();
    const okPhone = validatePhone();

    if (!(okName && okDept && okEvent && okEmail && okPhone)) {
      e.preventDefault();
    } else {
      e.preventDefault();
      alert('Registration submitted successfully.');
      form.reset();
```

```
      document.querySelectorAll('.msg').forEach((el) => {
        el.textContent = '';
        el.className = 'msg';
      });
    }
  });
 </script>
</body>
</html>
```

OUTPUT



Task 4: AI-Assisted E-Commerce Product Page

Scenario

A startup is developing a basic e-commerce product page to showcase products and manage simple cart functionality.

Tasks

1. Use Copilot to generate a grid-based product catalog using HTML and CSS.

2. Implement an “Add to Cart” feature using JavaScript.

3. Modify AI-generated code to include:

o Cart item counter at the top-right corner

o Visual feedback when a product is added

Expected Outcome

- **Functional product catalog**
- **Working cart counter**
- **Interactive user experience**

PROMPT

Create an e-commerce product page with a grid-based product catalog using HTML/CSS. Implement Add to Cart in JavaScript, include a cart item counter at the top-right, and show visual feedback when a product is added.

CODE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Task 4 - E-Commerce Product Page</title>
  <style>
    :root {
      --bg: #f4f6fb;
      --text: #1f2937;
```

```
  --primary: #2563eb;
  --ok: #16a34a;
  --card: #fff;
  --border: #dbe2f0;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Segoe UI", Tahoma, sans-serif;
  background: var(--bg);
  color: var(--text);
}

.topbar {
  background: #fff;
  border-bottom: 1px solid var(--border);
  position: sticky;
  top: 0;
  z-index: 3;
  padding: 12px 18px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

```
.brand {
  font-weight: 800;
  color: var(--primary);
}

.cart-wrap {
  position: relative;
  font-weight: 700;
}

.cart-counter {
  position: absolute;
  top: -10px;
  right: -14px;
  min-width: 22px;
  height: 22px;
  border-radius: 50%;
  background: #ef4444;
  color: #fff;
  display: grid;
  place-items: center;
  font-size: 0.78rem;
  padding: 0 5px;
}

.container {
  max-width: 1100px;
```

```
  margin: 0 auto;
  padding: 20px 16px 30px;
}

.products {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 14px;
}

.product-card {
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 14px;
  box-shadow: 0 8px 20px rgba(2, 6, 23, 0.06);
  transition: transform 0.2s ease;
}

.product-card:hover {
  transform: translateY(-4px);
}

.price {
  font-weight: 700;
  margin: 8px 0;
}
```

```
button {
  border: 0;
  border-radius: 8px;
  background: var(--primary);
  color: #fff;
  padding: 9px 12px;
  cursor: pointer;
  font-weight: 600;
  transition: transform 0.15s ease, background 0.2s ease;
}

button:hover {
  transform: scale(1.02);
}

.added {
  background: var(--ok);
}

.toast {
  position: fixed;
  right: 14px;
  bottom: 14px;
  background: #111827;
  color: #fff;
  padding: 10px 14px;
  border-radius: 8px;
  opacity: 0;
```

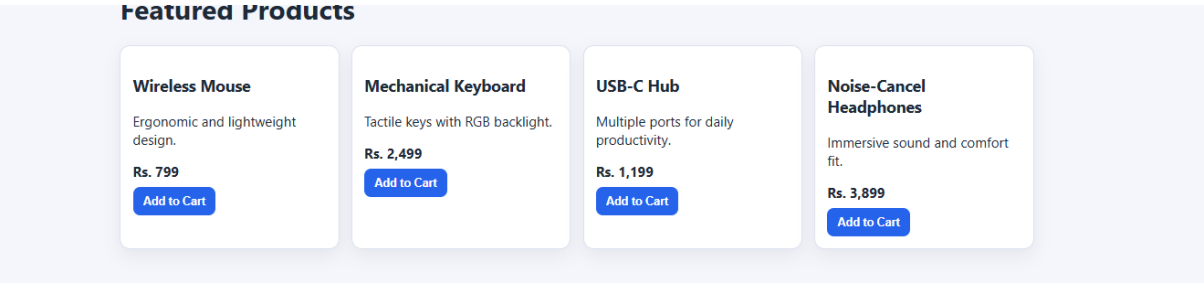
```
    transform: translateY(10px);
    transition: opacity 0.25s ease, transform 0.25s ease;
    pointer-events: none;
  }

  .toast.show {
    opacity: 1;
    transform: translateY(0);
  }
 </style>
</head>
<body>
 <!-- Simple Prompt: Create a grid-based product catalog with HTML/CSS and implement JavaScript Add to Cart with a top-right cart counter and visual feedback on add. -->

 <header class="topbar">
  <div class="brand">QuickKart</div>
  <div class="cart-wrap">
    Cart
    <span class="cart-counter" id="cartCount">0</span>
  </div>
 </header>

 <main class="container">
  <h1>Featured Products</h1>
  <section class="products" id="productGrid">
    <article class="product-card">
      <h3>Wireless Mouse</h3>
```

```
      <p>Ergonomic and lightweight design.</p>
      <p class="price">Rs. 799</p>
      <button type="button" class="add-btn">Add to Cart</button>
    </article>

    <article class="product-card">
      <h3>Mechanical Keyboard</h3>
      <p>Tactile keys with RGB backlight.</p>
      <p class="price">Rs. 2,499</p>
      <button type="button" class="add-btn">Add to Cart</button>
    </article>

    <article class="product-card">
      <h3>USB-C Hub</h3>
      <p>Multiple ports for daily productivity.</p>
      <p class="price">Rs. 1,199</p>
      <button type="button" class="add-btn">Add to Cart</button>
    </article>

    <article class="product-card">
      <h3>Noise-Cancel Headphones</h3>
      <p>Immersive sound and comfort fit.</p>
      <p class="price">Rs. 3,899</p>
      <button type="button" class="add-btn">Add to Cart</button>
    </article>
  </section>
</main>
```

```
<div class="toast" id="toast">Item added to cart</div>

<script>
  let cartTotal = 0;
  const countEl = document.getElementById('cartCount');
  const toast = document.getElementById('toast');

  function showToast(text) {
    toast.textContent = text;
    toast.classList.add('show');
    setTimeout(() => toast.classList.remove('show'), 1100);
  }

  document.querySelectorAll('.add-btn').forEach((button) => {
    button.addEventListener('click', () => {
      cartTotal += 1;
      countEl.textContent = String(cartTotal);

      button.classList.add('added');
      const originalText = button.textContent;
      button.textContent = 'Added';

      showToast('Product added to cart');

      setTimeout(() => {
        button.classList.remove('added');
        button.textContent = originalText;
      }, 700);
```

```
    });
  });
 </script></body>
</html>
```

OUTPUT



Task 5: Create a Responsive Web Page Layout

Instructions:

- **Design a basic web page layout with a header, main content area, and footer using HTML and CSS.**
- **Use AI to assist in generating responsive CSS for different screen sizes.**
- **Ensure the layout is clean and visually organized.**

Expected Output:

- **A responsive web page with:**
- **Header with navigation links**
- **Main section with placeholder text/images**
- **Footer with copyright or contact info**
- **Layout adapts correctly to desktop, tablet, and mobile screen sizes.**

PROMPT

Design a clean responsive webpage layout with a header and navigation links, main content with placeholder text/images, and a footer. Ensure it adapts to desktop, tablet, and mobile using responsive CSS.

CODE

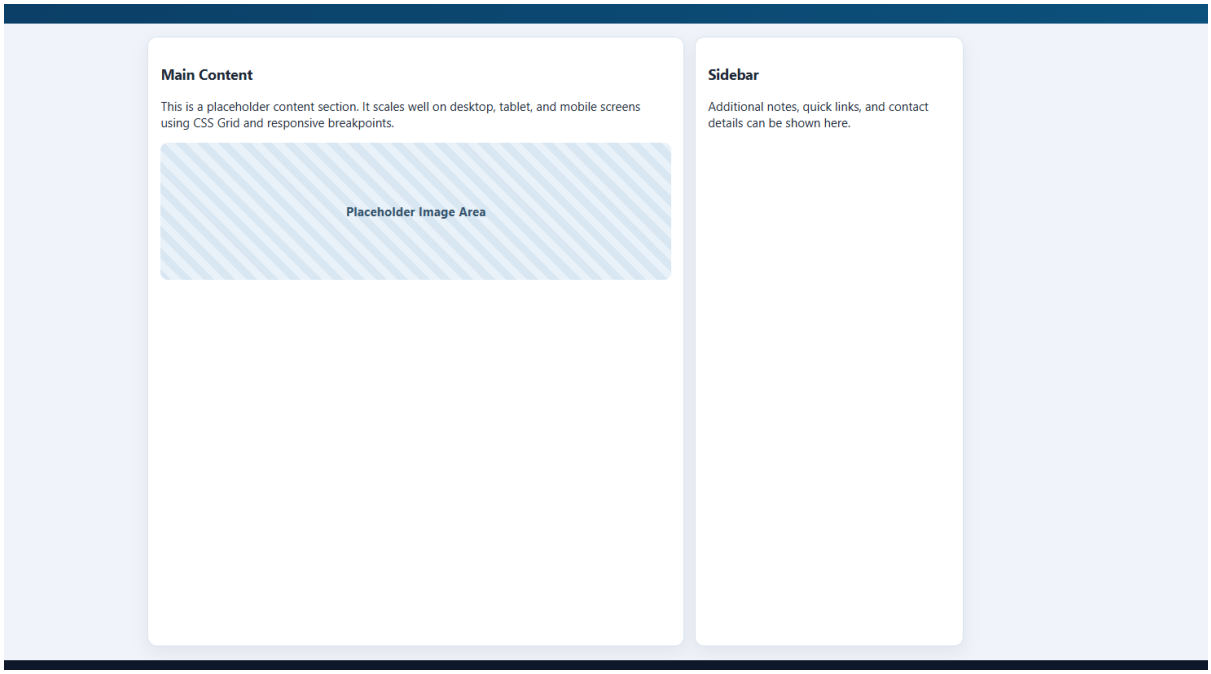
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Task 5 - Responsive Web Layout</title>
  <style>
    :root {
      --bg: #f0f4fa;
      --header: #0b3b62;
      --text: #1f2937;
      --surface: #ffffff;
      --border: #dbe4ee;
    }

    * {
      box-sizing: border-box;
    }

    body {
      margin: 0;
      font-family: "Segoe UI", Tahoma, sans-serif;
      background: var(--bg);
      color: var(--text);
      min-height: 100vh;
      display: flex;
      flex-direction: column;
```

```
}

header {
  background: linear-gradient(120deg, #0b3b62, #0c547f);
  color: #fff;
  padding: 14px 20px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 10px;
}

.nav {
  display: flex;
  gap: 14px;
  flex-wrap: wrap;
}

.nav a {
  color: #dff3ff;
  text-decoration: none;
  font-weight: 600;
}

.nav a:hover {
  color: #fff;
}
```

```
main {
  flex: 1;
  max-width: 1100px;
  width: 100%;
  margin: 18px auto;
  padding: 0 16px;
  display: grid;
  grid-template-columns: 2fr 1fr;
  gap: 14px;
}

.panel {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 16px;
  box-shadow: 0 8px 20px rgba(2, 8, 23, 0.06);
}

.image-placeholder {
  border-radius: 10px;
  height: 180px;
  background: repeating-linear-gradient(
    45deg,
    #d9e7f3,
    #d9e7f3 10px,
    #eaf2f9 10px,
```

```
    #eaf2f9 20px
  );
  display: grid;
  place-items: center;
  color: #35556f;
  font-weight: 700;
  letter-spacing: 0.2px;
}

footer {
  margin-top: auto;
  background: #0f172a;
  color: #cbd5e1;
  text-align: center;
  padding: 12px 10px;
  font-size: 0.93rem;
}

@media (max-width: 900px) {
  main {
    grid-template-columns: 1fr;
  }
}

@media (max-width: 560px) {
  header {
    align-items: flex-start;
  }
```

```
    .nav {
      gap: 10px;
    }
  }
 </style>
</head>
<body>
 <!-- Simple Prompt: Design a responsive webpage with header + nav links, main content with text/images, and footer; adapt layout for desktop, tablet, and mobile. -->

 <header>
  <h2>Responsive Layout Demo</h2>
  <nav class="nav">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Services</a>
    <a href="#">Contact</a>
  </nav>
 </header>

 <main>
  <section class="panel">
    <h3>Main Content</h3>
    <p>
      This is a placeholder content section. It scales well on desktop, tablet,
      and mobile screens using CSS Grid and responsive breakpoints.
    </p>
```

```
    <div class="image-placeholder">Placeholder Image Area</div>
  </section>

  <aside class="panel">
    <h3>Sidebar</h3>
    <p>Additional notes, quick links, and contact details can be shown here.</p>
  </aside>
 </main>

 <footer>
  Copyright 2026 | Contact: support@example.com
 </footer>
</body>
</html>
```

OUTPUT